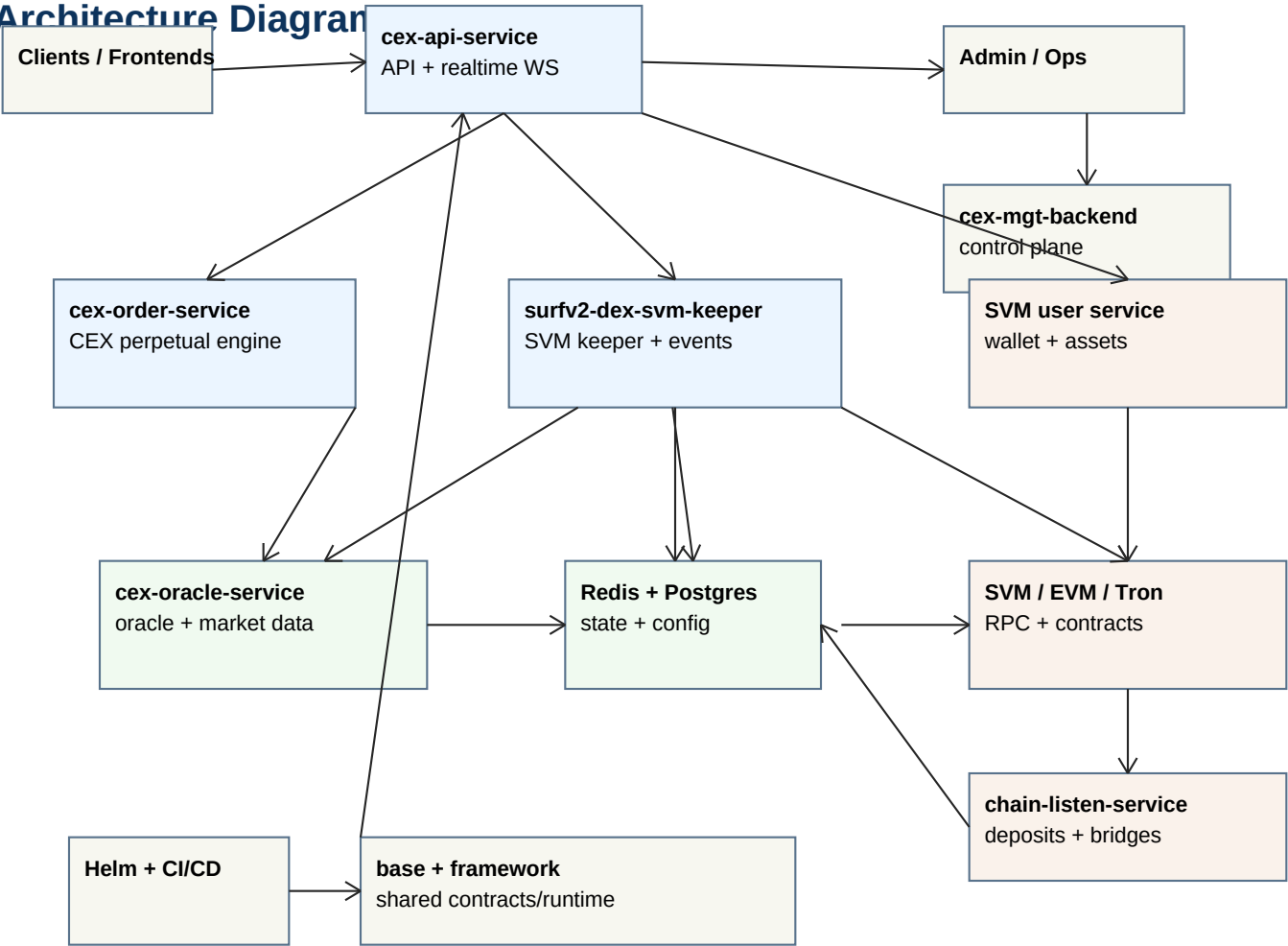


Turboflow Trading Engine Architecture Brief

Status: Draft | Date: 2026-05-14 | Source: Turboflow-HQ fbdf588 on main plus current local product repos

Executive view: one public API edge fronts two visible execution paths: CEX/off-chain perpetuals in cex-order-service, and Surf v2 SVM / event-contract trading in surfv2-dex-svm-keeper. Both rely on oracle data, shared base/framework contracts, Redis/Postgres, and Helm/Kubernetes deployment.

Architecture Diagram



Status note: dashed/missing-code services are documented as gaps in the report, especially cex-user-service and several Helm-only apps.

Part Status

API edge	Implemented, broad surface	cex-api-service fronts account, order, market, asset, pool, and realtime APIs.
CEX perpetual engine	Implemented, needs trace	cex-order-service owns off-chain orders, fills, positions, pools, risk, matcher, hedging.
SVM keeper	Implemented, high criticality	surfv2-dex-svm-keeper owns chain-backed execution, scanning, compensation, prediction/event orders.
Prediction/event contracts	Partially documented	Risk/blocking code exists; source contract repo is not visible, generated IDL is visible.
User asset plane	Implemented, split ownership	SVM user service, chain listener, and Fireblocks SDK cover wallet, assets, approvals, withdraws.
Oracle/market data	Implemented, ownership unclear	oracle-service is central; oracle-slippage overlaps or specializes in slippage/order books.
Admin/control	Implemented, mapping incomplete	platform-backend edits pair/risk/rule/sys config; field-to-consumer map is missing.
Shared foundation	Implemented, central coupling	base/framework define shared entities, RPC DTOs, config, chain, DB/Redis/HTTP helpers.
Delivery platform	Implemented, broader than code	code references cloned services plus missing services like cex-user-service and sign-service.

Key Trade Flows

- CEX perpetual order: client -> cex-api-service -> cex-order-service -> Postgres/Redis -> matcher/scheduler/risk/hedge -> client updates.
- Surf/SVM order: client -> cex-api-service -> surfv2-dex-svm-keeper -> SVM RPC/program -> scanner/compensation -> DB/asset effects.
- Prediction/event-contract order: API -> keeper prediction endpoint -> oracle freshness/slippage checks -> risk/blocking -> settlement/execution.
- Market data: exchanges/DEX sources -> cex-oracle-service -> Redis/DB/WebSocket/HTTP -> API, order service, keeper.
- Admin config: cex-mgt-backend/system endpoints -> sys_config/trade_config -> service caches and runtime behavior.

High Severity Gaps / Issues

- Missing source repos: runtime charts and API routes reference services not visible locally, especially cex-user-service.
- Implicit service contracts: base/rpc wrappers, route handlers, and DB config form contracts without one generated map.
- Runtime config governance: trading behavior depends on DB-backed sys_config and trade_config JSON values not captured as canonical production config.
- Prediction/event lifecycle: contract source, settlement state machine, oracle checks, risk gates, and asset effects need a dedicated trace.
- CEX order lifecycle: create/fill/position/liquidation/client-update path needs code-level documentation across API, RPC, matcher, and schedulers.

Medium Gaps

- Oracle/slippage overlap: cex-oracle-service and oracle-slippage share shape and may diverge in pricing or slippage logic.
- WebSocket ownership: API, oracle, order, and keeper expose event paths without a central topic and payload registry.
- Data model ownership: order/fill/position/pool/asset/cashbook tables are scattered across READMEs, models, DAOs, and migrations.
- Deployment map: Helm app inventory is broader than visible code, and ports/health/metrics/values ownership is not centrally extracted.

Likely Bottlenecks

- Oracle fan-out:** API, order service, and keeper all depend on oracle price/kline/ticker data. Stale or delayed oracle data can block or misprice both CEX and event-contract trading.
- Shared Redis/Postgres:** Order state, config, price caches, balances, and risk accounting share DB/Redis dependencies. Contention or cache drift can affect trading and balances.
- base coupling:** Most services import shared entities, enums, RPC DTOs, config, and chain helpers. Cross-repo changes have a wide blast radius.
- Keeper chain dependency:** SVM RPC, program/IDL compatibility, scanner, and compensation are critical. RPC degradation can leave orders in intermediate chain statuses.
- Runtime config opacity:** Behavior switches live in DB config, so incidents may be hard to reproduce from Git alone.

Recommended Next Actions

- Build service-contracts.md: inventory base/rpc clients, HTTP routes, ports, and runtime URLs.
- Build config-key-register.md: inventory sys_config and trade_config keys, owners, defaults, consumers, and admin fields.
- Trace cex-order-lifecycle.md: API request through CEX risk, DB writes, matcher, fills, positions, and client updates.
- Trace svm-keeper-lifecycle.md: API request through keeper, SVM submit/confirm/finish, scanner, compensation, and asset effects.
- Trace prediction-event-contracts.md: oracle checks, slippage, signal/band risk, settlement lifecycle, and contract artifacts.
- Map missing Helm apps to source repos or mark them external/runtime-only.

Source Basis

- Turboflow-HQ ops/repo-index.json and specs/trade-system/*.md at main commit fbdf588.
- Visible local repos: cex-api-service, cex-order-service, surfv2-dex-svm-keeper, surfv2-dex-svm-user-service, cex-oracle-service, oracle-slippage, cex-chain-listen-service, cex-mgt-backend, base, framework, amber-helm-charts, amber-cicd-lib.
- Risk-management wiki is present at current main and was referenced for overlapping risk-control context.
- Limitation: non-visible runtime services referenced by Helm/API routing were treated as gaps, not inferred as implemented.